

Lab 4: Pointers and Dynamic Memory Allocation

Total Points: 50

Objectives

- Implement dynamic memory allocation
- Recognize different issues in DMA
- Using dynamic arrays in writing function
- Passing parameters as pointers in functions
- Returning pointers from a function

Preparation

Review the quiz for chapters 11 on <http://liveexample.pearsoncmg.com/liang/cpp3e/toc.html>. Only answer questions **between 1 to 34**.

Part 1 - Analysis and Debugging (answers.pdf)

Question 1 - [15 points] What is the output of the following code

A)

```
void foo(char *px) {
    px++;
}
int main() {
    char s1[] = "ABCD";
    foo(s1);
    cout << s1 << endl;
}
```

B)

```
void foo(char *px) {
    (*px)++;
}
int main() {
    char s1[] = "ABCD";
    foo(s1);
    cout << s1 << endl;
}
```

C)

```
void foo(char *&px) {
    px++;
}
int main() {
    char s1[] = "ABCD";
    foo(s1);
    cout << s1 << endl;
}
```

D)

```
void foo(char *& px) {
    px++;
}
int main() {
    char tmp[] = "ABCD";
```

```

    char *s1 = tmp;
    foo(s1);
    cout << s1 << endl;
}

```

E)

```

void foo(char * & px){
    px++;
}
int main(){
    char tmp[] = "ABCD";
    char *s1 = tmp;
    foo(s1);
    s1[1] = 'Z';
    cout << tmp << endl;
    cout << s1 << endl;
}

```

Question 2 - [12 points] Identify the memory problem in the following code:

A)

```

int* p = new int(10);
p = new int(20);

```

B)

```

int* p = new int(5);
delete p;
cout << *p;

```

C)

```

int* arr = new int[10];
delete arr;

```

D)

```

void func() {
    int* x = new int(50);
}

```

E)

```

int* p = new int(10);
int* q = p;
delete p;
cout << *q;

```

F)

```

int* p = new int(5);
delete p;
p = nullptr;

```

Question 3 – [3 points]

- A) Which is more dangerous: memory leak or dangling reference? Why?
- B) Can a program work correctly even if it has a memory leak? Explain.

Part 2 – Programming (testProgram.cpp)

Question 4 - [20 points] Write a function with the exact prototype for each task listed below:

// A: a function that returns a copy of an array (data) with the given
// size

```
double* copyArray (double data [], int size);
```

// B: a function that returns the reverse of an array (data) with the
// given size

```
int* reverse (int* data, int size);
```

// C: a function that returns a list of all prime numbers less than a
// given number. it also returns the size of the new list.

// prime number is a number that is only divisible by 1 and itself.

```
int* primes(int number, int& size);
```

// D: a function that returns the maximum and minimum of a list with a
// given size

```
void minMax(double *list, int size, double *min, double *max);
```

// E: write a main function to call the above functions in a test program
// to test your functions.

Submission

Submit a zip file including **testProgram.cpp** and **answers.pdf** in the Brightspace before the due date.